Unit 10: Parallel Schemes: Merge Sort, Matrix Multiplication

Unleashing the Power of Multithreading

Ivan Temesvari
Distinguished Professor of Mathematics and Computer Science
Oakton College

Introduction to Parallelism

The Need for Speed: Embracing Parallelism

Moore's Law and its limitations: The slowdown in single-core performance improvements

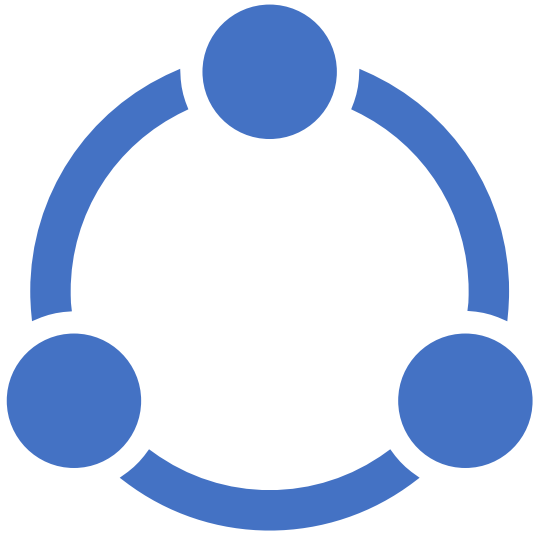
Parallel Computing: Leveraging multiple cores/processors to execute tasks concurrently.

Benefits: Improved performance, efficiency, and scalability for computationally intensive tasks.

Multithreaded Algorithms

Divide and Conquer: Multithreading in Action

- **Definition:** Breaking down a problem into smaller subproblems that can be solved independently.
- **Key Concepts:** Threads, synchronization, load balancing.
- **Advantages:** Exploits parallelism in modern hardware, enhances performance on multi-core systems.



Case Study: Merge Sort
Sorting at Scale: The Power of Parallel Merge Sort

Algorithm Overview: Recursively dividing the array, sorting subarrays, and merging them in parallel.

Challenges: Efficiently merging sorted subarrays, managing threads, and avoiding race conditions.

Real-world Application

Large-scale data processing: Companies like Google and Facebook handle petabytes of data daily. Parallel merge sort allows them to sort and analyze this data quickly, enabling real-time search results, personalized recommendations, and targeted advertising.

Database sorting: Databases rely heavily on sorting for indexing and retrieving data. Parallel merge sort accelerates query execution, improving database performance and responsiveness.

Scientific simulations: Scientific research heavily relies on simulations to model complex phenomena. Parallel computing enables researchers to run larger and more detailed simulations, leading to new discoveries and breakthroughs.

Case Study: Matrix Multiplication

Image Processing:

Image processing involves numerous matrix operations (e.g., convolution, blurring, edge detection).

Parallel matrix multiplication enables real-time image editing, video processing, and computer vision applications.

Machine Learning:

Neural networks are built on matrix operations, and training them on large datasets requires significant computational power.

Parallel matrix multiplication accelerates the training process, enabling faster development and deployment of AI models.

Parallelism in Gaming and Graphics:

Game engines leverage parallel computing to render complex scenes, simulate physics, and provide immersive experiences.

Parallelism enables realistic graphics, AI-driven characters, and large-scale multiplayer interactions.

Parallelization Beyond Multithreading: Exploring Parallelization Techniques

Task Parallelism: Distributing independent tasks across multiple cores.

Data Parallelism: Performing the same operation on different parts of a large dataset concurrently.

Challenges: Identifying parallelism, communication overhead, load balancing.

Case Study: Matrix Multiplication
Crunching Numbers: Parallel Matrix Multiplication

- **Algorithm Overview:** Distributing matrix elements and computations across multiple cores.
- **Optimization Techniques:** Loop tiling, cache blocking, and utilizing specialized hardware (e.g., GPUs).
- **Real-world Application:** Image processing, machine learning, scientific simulations.

Conclusion
Parallelism: The Future of Computing

- Parallel computing is essential to tackle the growing complexity of modern problems
- Multithreading and parallelization techniques enable us to harness the full potential of multi-core systems.
- Understanding these concepts empowers us to develop efficient and scalable solutions.